



指令系统

指令格式

分类

- 动作类
 - 传送
 - 运算
 - I/O
- 控制
 - 循环
 - 移位
 - 停机

操作码：该指令应进行什么性质的操作

- n位操作码最多描述 2^n 种操作
 - 定长：便于译码，扩展性差
 - 变长：能缩短指令平均长度

地址码：各种数据的来源/存放地址

- 操作数的地址
- 操作结果的存储地址
- 下一条指令的地址

零地址指令

格式： *OPCODE*

只有操作码，没有地址码

- 无需操作数，如停机指令
- 或：操作数是默认的

一地址指令

格式： *OPCODE* + *A*

- 只给出一个地址，既是操作数的地址，又是操作结果的存储地址
- A 可以是存储器地址或寄存器名
- 或： A 给出一个操作数地址，另一个操作数在一个默认寄存器（累加器）中，且运算结果写回累加器

二地址指令

格式： $OPCODE + A_1 + A_2$

- 两个地址码给出两个操作数的地址
- 一般 A_2 给出存储操作结果的地址

三地址指令

格式： $OPCODE + A_1 + A_2 + A_3$

- A_1, A_2 给出两个操作数的地址
- A_3 给出存储操作结果的地址

多地址指令

格式： $OPCODE + A_1 + A_2 + \dots + A_n$

- 一般用于批处理
- 如字符串、向量、矩阵运算

指令字长度

- 等长：取指快、译码简单，但存储空间存在浪费
- 变长：可提高编码效率，但控制较复杂
- 通常是字节的整数倍

指令格式选取考虑的因素：

1. 指令长度应尽可能的短，短指令比长指令好
2. 指令长度与机器字长应是字符长度的整倍数
3. 指令中操作码字段应有足够的位数，使之能表示指令系统中全部操作。
4. 指令中地址段的位数要足够长

数据存放方式

如数据：00000000 00000001 00010000
00000011

大端序和小端序的存储方式

32位字存储单元的最低2位地址	小端序	大端序
第00单元	00000011	00000000
第01单元	00010000	00000001
第10单元	00000001	00010000
第11单元	00000000	00000011

寻址方式

指令寻址

确定下一条将要执行的指令的地址

- 1. 顺序寻址：PC=PC+4(1)
- 2. 跳跃寻址：当前指令指定，地址送PC

操作数寻址

确定数据的来源/存储地址

- 1. 直接寻址：数据存放在存储器中，指令给出存储器地址
- 2. 寄存器寻址：数据存放在寄存器中，指令给出寄存器编号
- 3. 基址寻址：数据存放在存储器中
 - 设有基址寄存器，存放基址
 - 指令给出偏移量
 - 数据地址由二者相加所得
- 4. 变址寻址：数据存放在存储器中
 - 设有变址寄存器（可变，如累加器）
 - 指令给出基准地址（类似基址）
 - 数据地址由二者相加所得
 - 一般用于数组遍历等（不断改变变址寄存器中的值）

5. 间接寻址：数据存放在存储器中
 - 数据**地址**存放在寄存器中，指令给出该寄存器编号
 - 数据**地址**存放在存储器中，指令给出该存储器地址
 - 第一次读出地址，第二次由读出的地址读数据
 - 可多次间址（一般只允许一次）
6. 立即数寻址：地址码段直接给操作数，不用寻址

指令系统的实现

- **完备性**：指令系统丰富、功能齐全、使用方便；应该包括：
 - 数据传送指令
 - 算术、逻辑运算指令
 - 程序控制指令
 - 输入/输出指令
- **有效性**：用该指令系统所编写的程序能够高效率地运行
 - 程序占据存储空间小、执行速度快
- **规整性**
 - **对称性**：所有的寄存器和存储器单元都可同等对待，所有的指令都可使用各种寻址方式
 - **匀齐性**：一种操作性质的指令可以支持各种数据类型
 - **一致性**：指令长度和数据长度有一定的关系，以方便处理和存取
- **兼容性**
 - 向上兼容：低档机上运行的软件可以在高档机上运行
 - 向后兼容：先生产机器上运行的软件可以在后生产的机器上运行

指令系统的优化设计

操作码的优化设计

Huffman编码

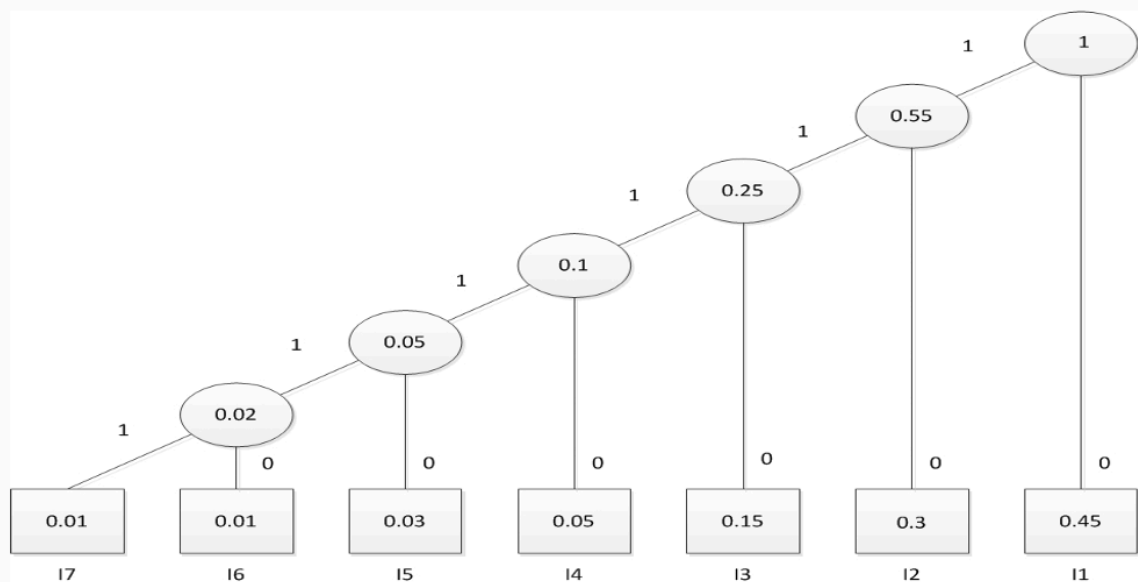
1. **排序**：将所有指令按操作码出现概率 **从低到高** 排列。
2. **合并最小节点**：
 - 选取 **两个概率最小的节点** 合并，生成一个新节点（概率=两者之和）。
 - 将新节点放回集合，并 **重新按概率排序**。

3. **重复合并**：重复步骤2，直到所有节点合并为 **一个根节点**（概率=1）。

4. **分配编码**：

- 每个节点的两个分支分别标记为 0 和 1。
- 从根节点到目标节点的路径代码组合，即为该指令的 **Huffman编码**。

指令	I1	I2	I3	I4	I5	I6	I7
使用频度	0.45	0.30	0.15	0.05	0.03	0.01	0.01



地址码的优化设计

1. 个数的选择：主要标准有
 - 对于程序存储量的计算
 - 对于程序执行速度的计算
2. 缩短单个地址码长度的方法
 - 间接寻址
 - 变址寻址
 - 寄存器间址寻址

CISC和RISC的区别

以下是 **CISC** 和 **RISC** 主要特点的对比表格：

特点	CISC (复杂指令集计算机)	RISC (精简指令集计算机)
指令系统	庞大，指令功能复杂，格式多，寻址方式多	精简，指令数量少，格式固定，寻址方式少
指令执行周期	绝大多数指令需要多个机器周期完成	绝大多数指令只需1个机器周期完成
存储器访问	几乎所有指令都可以直接访问存储器	只有 <code>Load/Store</code> 指令可以访问存储器，其余操作基于寄存器
控制方式	主要采用 微程序控制 (Microprogramming)	主要采用 硬布线控制 (Hardwired Control)
寄存器数量	有少量专用寄存器	通用寄存器数量多，减少访存开销
编译优化	难以用优化编译技术生成高效目标代码	易于优化编译，提高执行效率
典型架构	x86 (Intel, AMD)	ARM, MIPS, RISC-V